

C Programming – Brushing Up Questions: Answer Key

Q1. Define a variable. Write the syntax to declare an integer variable age and initialize it to 20.

Answer:

A variable is a named location in computer memory that is used to store a value, which can be changed (varied) during the execution of a program. Every variable in C must be declared with a data type before it is used.

Syntax to declare and initialize an integer variable named age with value 20:

```
int age = 20;
```

Q2. List the basic data types in C along with their format specifiers used in printf/scanf.

Answer:

- int – Integer (whole numbers) — format specifier: %d
- float – Single-precision floating point (decimal numbers) — format specifier: %f
- double – Double-precision floating point — format specifier: %lf (scanf) / %f (printf)
- char – Single character — format specifier: %c

(Note: printf accepts %f for both float and double due to default argument promotion, but scanf requires %lf for double and %f for float.)

Q3. What is the difference between int, float, and char data types? Give one real-life example variable for each.

Answer:

- int – stores whole numbers (no decimal part). Example: int age = 25; (a person's age)
- float – stores real numbers with a fractional/decimal part. Example: float price = 99.99; (price of an item)
- char – stores a single character enclosed in single quotes. Example: char grade = 'A'; (a student's grade)

Q4. Write a C program to declare two integer variables a and b, take input from the user, and print their sum.

Answer:

```
#include <stdio.h>

int main() {
    int a, b, sum;

    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;
    printf("Sum = %d\n", sum);

    return 0;
}
```

Q5. What is the output of the following code?

```
int x = 5;
printf("%d", x);
```

Answer:

Output: 5

The variable x is initialized to 5, and %d prints its integer value as-is.

Q6. Write the correct format specifier for: (a) integer (b) float (c) character (d) double (e) string

Answer:

- (a) integer → %d
- (b) float → %f
- (c) character → %c
- (d) double → %lf (in scanf; %f is also accepted by printf)
- (e) string → %s

Q7. Differentiate between = (assignment operator) and == (equality operator) with an example.

Answer:

- = is the assignment operator. It assigns the value on its right-hand side to the variable on its left-hand side. Example: a = 5; → assigns 5 to a.
- == is the equality (relational/comparison) operator. It compares two values and returns 1 (true) if they are equal, or 0 (false) otherwise. Example: if (a == 5) → checks whether a is equal to 5.

A common beginner mistake is writing if (a = 5) when if (a == 5) was intended — this assigns 5 to a instead of comparing.

Q8. Write a program to check whether a number entered by the user is positive or negative using if-else.

Answer:

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num >= 0) {
        printf("%d is positive\n", num);
    } else {
        printf("%d is negative\n", num);
    }

    return 0;
}
```

Note: With only two branches (if-else), 0 is grouped with the positive case here. If a separate case for zero is required, a third condition (num == 0) should be added.

Q9. What is the purpose of scanf()? Write its syntax to read an integer into variable n.

Answer:

scanf() is a standard input function in C used to read formatted data entered by the user (typically from the keyboard) and store it into variables. It requires the address-of operator (&) before the variable name (except for strings/arrays).

Syntax to read an integer into variable n:

```
scanf("%d", &n);
```

Q10. Evaluate the following expressions (assume a = 10, b = 3): (a) a + b (b) a % b (c) a / b (d) a * b

Answer:

- (a) $a + b = 10 + 3 = 13$
- (b) $a \% b = 10 \% 3 = 1$ (remainder of integer division)
- (c) $a / b = 10 / 3 = 3$ (integer division truncates the decimal part)
- (d) $a * b = 10 * 3 = 30$

Q11. Write a program using if statement to check if a number is even or odd.

Answer:

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 == 0)
        printf("%d is even\n", num);
    else
        printf("%d is odd\n", num);

    return 0;
}
```

Q12. What will be the value of c after execution?

```
int a = 4, b = 2, c;
c = a > b;
```

Answer:

c = 1

The expression $a > b$ ($4 > 2$) is a relational comparison that evaluates to true, and in C, true is represented as the integer 1 (false would be 0). This value 1 is then assigned to c.

Q13. Identify the type of operator used in each case: +, &&, <=, ++, =

Answer:

- + → Arithmetic operator (addition)
- && → Logical operator (logical AND)
- <= → Relational operator (less than or equal to)
- ++ → Unary / Increment operator
- = → Assignment operator

Q14. Write a program to find the largest of two numbers using if-else.

Answer:

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("%d is the largest\n", a);
    else
        printf("%d is the largest\n", b);

    return 0;
}
```

Q15. Write a program to find the largest of three numbers using only nested if statements (an if inside another if, no else if ladder).

Answer:

```
#include <stdio.h>

int main() {
    int a, b, c;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b) {
        if (a > c)
            printf("%d is the largest\n", a);
        else
            printf("%d is the largest\n", c);
    } else {
        if (b > c)
            printf("%d is the largest\n", b);
        else
            printf("%d is the largest\n", c);
    }

    return 0;
}
```

Q16. Write a program using switch statement that takes a number from 1–7 and prints the corresponding day name (Monday–Sunday).

Answer:

```
#include <stdio.h>

int main() {
    int day;

    printf("Enter a number (1-7): ");
    scanf("%d", &day);

    switch (day) {
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
        case 5: printf("Friday\n"); break;
        case 6: printf("Saturday\n"); break;
        case 7: printf("Sunday\n"); break;
        default: printf("Invalid input\n");
    }

    return 0;
}
```

Q17. What is the purpose of the break statement inside a switch block? Write a short example showing what happens if it is left out.

Answer:

The break statement terminates execution of the switch block as soon as the matching case has been executed. Without break, control does not exit the switch after a matching case — it "falls through" and continues executing the statements of all the subsequent cases (and the default), regardless of whether their labels match, until a break is encountered or the switch block ends.

Example without break:

```
#include <stdio.h>

int main() {
    int day = 2;

    switch (day) {
        case 1: printf("Monday\n");
        case 2: printf("Tuesday\n");
        case 3: printf("Wednesday\n");
        default: printf("Invalid\n");
    }

    return 0;
}
```

Output (day = 2), since case 2 matches but there is no break:

Tuesday
Wednesday
Invalid

Q18. Write a menu-driven switch program that performs basic arithmetic (+, -, *, /) based on an operator character entered by the user.

Answer:

```
#include <stdio.h>

int main() {
    char op;
    float a, b, result;

    printf("Enter two numbers: ");
    scanf("%f %f", &a, &b);

    printf("Enter an operator (+, -, *, /): ");
    scanf(" %c", &op);

    switch (op) {
        case '+':
            result = a + b;
            printf("Result = %.2f\n", result);
            break;
        case '-':
            result = a - b;
            printf("Result = %.2f\n", result);
            break;
        case '*':
            result = a * b;
            printf("Result = %.2f\n", result);
            break;
        case '/':
            if (b != 0) {
                result = a / b;
                printf("Result = %.2f\n", result);
            } else {
                printf("Error: Division by zero\n");
            }
            break;
        default:
            printf("Invalid operator\n");
    }

    return 0;
}
```

Note: A leading space before %c in scanf(" %c", &op) is used to skip any leftover whitespace/newline from the previous input.

Q19. Write a program using nested if to check whether a number is positive, and if positive, further check whether it is even or odd.

Answer:

```
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0) {
        if (num % 2 == 0)
            printf("%d is positive and even\n", num);
        else
            printf("%d is positive and odd\n", num);
    } else {
        printf("%d is not positive\n", num);
    }

    return 0;
}
```